

Documentação do Sistema – Revisaí

1. Visão Geral

O Revisaí é um sistema web/mobile para gerenciamento de veículos pessoais ou corporativos.

Ele centraliza informações de manutenção, despesas, alertas e compartilhamento de veículos entre usuários com diferentes permissões.

2. Objetivos

- Facilitar o acompanhamento de manutenções preventivas e corretivas.
- Controlar despesas veiculares com histórico detalhado.
- Emitir alertas automáticos baseados em tempo ou quilometragem.
- Gerar checklists e relatórios exportáveis.
- Permitir compartilhamento seguro de veículos com outros usuários.

3. Funcionalidades Principais

- Cadastro/login de usuários (incluindo integração com Google OAuth).
- Cadastro de veículos com validação de placa.
- Inclusão de motoristas autorizados.
- Registro de despesas e manutenções (com OCR para notas fiscais).
- Sistema de alertas configuráveis e inteligentes.
- Histórico e relatórios filtráveis, exportáveis em PDF.
- Compartilhamento de veículo por e-mail ou código.
- Painel de checklist de viagem.

4. Diagramas e Relações

4.1 Caso de Uso

- Atores: Proprietário do veículo e usuário convidado.
- Proprietário gerencia todos os dados; convidado tem acesso conforme permissão.
- Casos de uso abrangem cadastro, manutenção, alertas, despesas e compartilhamento.

4.2 Atividade

- Fluxos principais, como compartilhamento e cadastro.
- Representa decisões, validações e possíveis ramificações no sistema.

4.3 Classe

- Classes: Usuário, Veículo, Manutenção, Alerta, Despesa, Histórico, Checklist, Compartilhamento.
- Relações diretas:
- Usuário 1..* → Veículo
- Veículo → Manutenção, Alerta, Despesa
- Checklist vinculado a Viagem.

4.4 Estado

- Ciclo de vida dos alertas: Criado → Ativo → Atendido / Expirado / Cancelado.
- Define respostas automáticas do sistema conforme interações e prazos.

4.5 Sequência

- Vários fluxos detalhados:
- Compartilhamento de veículo.
- Cadastro (com e sem Google).
- Adição de veículos e motoristas autorizados.
- Geração de checklist e exportação PDF.
- Registro de despesas e manutenção com OCR.
- Emissão e confirmação de alertas.

4.6 Arquitetura

- Camadas:
- Apresentação (Web/App)
- Negócio (lógica e regras)
- Dados (persistência no banco)
- Segue padrão MVC, garantindo separação de responsabilidades.

4.7 Componentes

- Módulos independentes:
- Gestão de usuários.
- Cadastro de veículos.
- Controle de manutenção e despesas.
- Sistema de alertas.
- Compartilhamento de veículos.
- Interfaces bem definidas para integração.

5. Tecnologias Utilizadas

- Frontend: React ou Flutter.
- Backend: Java Spring Boot.
- Banco de Dados: MySQL.
- Integrações: Google Maps API, OCR.
- Controle de Versão: GitHub.

6. Considerações para Desenvolvimento

- Usar autenticação segura (JWT/OAuth).
- Criar API REST padronizada.
- Garantir que alertas e checklists sejam gerados com base em regras configuráveis.
- Implementar controle de permissões granulares para compartilhamento.